

Design Ride Sharing

Uber · Ola · Rapido – "drivers స్థానం పంపుతారు,
దగ్గరివాడిని rider కి కలుపుదాం" అని అందరూ
మొదలుపెడతారు.

మూడు విరుపులు. మొదటిది – driver స్థానాలని database lo రాష్ట్ర సెకనుకి
2,50,000 writes, మరియు వాటిలో **99.97% అనవసరం**. రెండోది – "దగ్గరివాడిని
వెంటనే" అనే నియమం drivers కొరతగా ఉన్నప్పుడు pickup దూరాన్ని **రెట్టింపు**
చేస్తుంది. మూడోది – drivers తిరస్కరిస్తుంటే rider **29 సెకన్లు** ఎదురుచూస్తాడు,
మరియు దాని పరిష్కారం **2.5 మంది drivers ని మోసం చేస్తుంది**.

ఈ doc ఎలా చదవాలి

పక్కన editor తెరిచి కలిసి రాయండి. ఇక్కడి ప్రతి output, ప్రతి శాతం నిజంగా `node` lo run చేసినదే.

LLD DEEP 11 (FOOD DELIVERY) LO GEO INDEX ఉంది - దాన్ని ఇక్కడ మళ్ళీ రాయను

అక్కడ GeoIndex grid, kmPerDegLon bug, మరియు 12x వేగం - అన్నీ కొలిచాం. "దగ్గరివాళ్ళని ఎలా వెతకాలి" అనే ప్రశ్నకి జవాబు అక్కడ ఉంది, మరియు అది ఇక్కడా అదే (geohash లేదా H3, ఒక వాక్యంలో).

ఈ doc అడిగేది వేరే ప్రశ్న, మరియు అది geo కాదు:

"దగ్గరివాడు దొరికాడు. ఇప్పుడు ఏమి చెయ్యాలి?"

ఎందుకంటే మిగతా అన్ని systems lo మీరు ఒక వనరుని కేటాయిస్తారు. ఇక్కడ మీరు ఒక మనిషిని అడుగుతారు - మరియు అతను వద్దు అనొచ్చు.

అదే ఈ problem ని ప్రత్యేకం చేస్తుంది. §7 lo "వెంటనే కేటాయించడం" తప్పని కొలుస్తాం. §10 lo "అడిగితే వద్దంటాడు" అనేదాని ఖర్చు కొలుస్తాం.

మరియు రెండింటికీ జవాబు ఒకే ఆకారంలో ఉంటుంది: **ఎప్పుడు ఆగాలో తెలియడం.**

విషయ సూచిక (Table of Contents)

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. అడిగింది ఏమిటి – మరియు ఇది కేటాయింపు కాదు, ఒక ప్రతిపాదన
2. Clarifying questions – ప్రతిదాని వెనక ఉన్న ఉద్దేశం

Part 2 – మొదటి విరుపు: స్థానాల వరద

3. Step – ప్రతి 4 సెకన్లకీ ప్రతి driver
4. మొదటి విరుపు – సెకనుకి 2,50,000 writes

Part 3 – రెండో విరుపు: వెంటనే కేటాయించడం

5. Step – దగ్గరివాడిని వెంటనే
6. రెండో విరుపు – అత్యాశ ఎప్పుడు తప్పు అవుతుంది
7. Step – ఒక కిటికీ, ఒక గుంపు

Part 4 – మూడో విరుపు: driver వద్దంటాడు

8. Step – offer పంపి ఎదురుచూడటం
9. మూడో విరుపు – 29 సెకన్లు, లేదా 2.5 మోసాలు

Part 5 – పూర్తి system

Part 6 – Interview lo

11. ఇదంతా 45 నిమిషాల్లో ఎలా చెప్పాలి
12. నోటితో చెప్పాల్సిన English script
13. Follow-ups – surge, pooling, ETA
14. ఏమి నేర్చుకున్నాం

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. అడిగింది ఏమిటి – మరియు ఇది కేటాయింపు కాదు, ఒక ప్రతిపాదన

Riders ప్రయాణం అడుగుతారు. Drivers చుట్టూ తిరుగుతున్నారు. వాళ్ళని కలపాలి.

సాంకేతికంగా ఇది ఒక **matching problem**, మరియు దానికి గణితం ఉంది. కానీ ఒక్క తేడా ఆ గణితాన్ని పనికిరానిదిగా చేస్తుంది:

CONNECTION POOL కీ ఈ PROBLEM కీ తేడా

LLD Deep 24 lo ఒక connection ని `borrow()` చేశాం. అది వద్దనలేదు.

LLD Deep 26 lo ఒక thread కి పని ఇచ్చాం. అది వద్దనలేదు.

ఇక్కడ మీరు ఒక driver కి ride **ఇవ్వలేరు** – మీరు అతన్ని **అడగగలరు**. మరియు §9 lo కొలుస్తాం: అతను 80% సార్లు **వద్దు** అనిచ్చు.

అంటే మీ "optimal assignment" ఒక **సూచన** మాత్రమే. నిజమైన కేటాయింపు drivers చేతిలో ఉంది.

అనుకుందాం: 10 లక్షల drivers · ప్రతి 4 సెకన్లకీ ఒక స్థానం update · ఒక నగరంలో వేలమంది riders.

2. Clarifying questions – ప్రతిదాని వెనక ఉన్న ఉద్దేశం

ప్రశ్న	ఎందుకు అడుగుతున్నాం
Driver స్థానాల చరిత్ర కావాలా, లేక ఇప్పటి స్థానం చాలా?	చరిత్ర అవసరం లేకపోతే 99.97% writes పోతాయి – §4

ప్రశ్న	ఎందుకు అడుగుతున్నాం
Rider ఎంతసేపు ఎదురుచూడగలడు?	ఇదే §7 యొక్క కీటికీ. 5 సెకన్లు ఆగితే pickup 40% దగ్గర అవుతుంది
Drivers ఎంత శాతం అంగీకరిస్తారు?	ఈ ఒక్క సంఖ్య §9 మొత్తాన్ని నిర్ణయిస్తుంది – 70% vs 20% పూర్తిగా వేరే design
ఒకేసారి ఎంతమంది drivers కి పంపొచ్చు?	ఎక్కువ పంపితే వేగం, కానీ drivers మోసపోతారు – §9
Supply ఎంత గట్టిగా ఉంటుంది?	§7 – drivers ఎక్కువగా ఉంటే కీటికీ పనికిరాదు
Trip మధ్యలో network పోతే?	State machine ఎక్కడ ఉంటుంది – ఫోన్ lo నా, server lo నా?
Driver ఒకేసారి రెండు rides తీసుకోగలడా?	Pooling ఉంటే అవును – §13

ఒక ప్రశ్న మీ మొత్తం MATCHING DESIGN ని నిర్ణయిస్తుంది

"Peak hour lo drivers ఎంత కొరతగా ఉంటారు?"

ఎందుకంటే §7 lo కొలుస్తాం: drivers ఎక్కువగా ఉన్నప్పుడు తెలివైన matching **0.9%** మాత్రమే మెరుగుపరుస్తుంది – అంటే దాదాపు వృథా.

Drivers కొరతగా ఉన్నప్పుడు అదే matching **50.7%** మెరుగుపరుస్తుంది.

కాబట్టి "ఎప్పుడూ తెలివిగా match చెయ్యి" అనేది తప్పు జవాబు. సరైనది – **కొరత ఉన్నప్పుడు మాత్రమే.**

Part 2 – మొదటి విరుపు: స్థానాల వరద

3. Step – ప్రతి 4 సెకన్లకీ ప్రతి driver

Matching కి drivers ఎక్కడున్నారో తెలియాలి. కాబట్టి ప్రతి driver app ప్రతి కొన్ని సెకన్లకీ తన స్థానం పంపుతుంది.

```
app.post('/location', (req) => {  
  db.insert('driver_locations', {  
    driverId: req.driverId, lat: req.lat, lng: req.lng, at: Date.now()  
  });  
});
```

సూటిగా ఉంది. ఒక్కో update 40 bytes. లెక్క వేద్దాం.

4. మొదటి విరుపు – సెకనుకీ 2,50,000 writes

1 మిలియన్ drivers · ప్రతి 4 సెకన్లకీ ఒక update

సెకనుకీ updates : 2,50,000
రోజుకి : 21.6 బిలియన్

ఒక్కో update 40 bytes → సెకనుకీ 10.0 MB · రోజుకి 0.9 TB

సెకనుకీ 2,50,000 DATABASE WRITES – మరియు వాటిలో ఏదీ ఎవరూ చదవరు

ఆ writes అన్నీ ఒకే ప్రశ్నకి జవాబు ఇవ్వడానికీ: **"driver 47 ఇప్పుడు ఎక్కడ ఉన్నాడు?"**

కానీ ఆ ప్రశ్నకి జవాబు **అతని చివరి update** మాత్రమే. మిగతా 21.6 బిలియన్ rows ఎవరూ ఎప్పుడూ చదవరు.

మరియు ఒక ముఖ్యమైన విషయం: **ఆ డేటా పోయినా ఫరవాలేదు.** Server restart అయితే, 4 సెకన్లలో ప్రతి driver మళ్ళీ చెప్తాడు.

ఇదే ఈ విరుపు యొక్క కేంద్రం, మరియు అది ఒక్క వాక్యంలో ఉంది:

స్థానం ఒక రికార్డు కాదు - అది ఒక క్షణిక స్థితి

Database lo మనం జరిగిన విషయాలని నిల్వ చేస్తాం: ఒక trip, ఒక payment, ఒక rating. అవి శాశ్వతమైన నిజాలు.

"Driver ఎక్కడ ఉన్నాడు" అనేది ఒక శాశ్వతమైన నిజం కాదు - అది **4 సెకన్లలో పాతది** అవుతుంది, మరియు **పోయినా తిరిగి వస్తుంది**.

అలాంటిది database lo ఉండాల్సిన అవసరం లేదు. అది **memory lo** ఉండాలి.

```
// §4 - స్థానం ఒక క్షణిక స్థితి*, ఒక రికార్డు కాదు. memory lo మాత్రమే.  
updateLocation(id, x, y) {  
  let d = this.#drivers.get(id);  
  if (!d) { d = { id, x, y, busyWith: null }; this.#drivers.set(id, d); }  
  else { d.x = x; d.y = y; }  
  this.stats.located++;  
  return true;  
}
```

ఈ updates తో ఏమి చెయ్యాలి?

విధానం	write ఖర్చు	నిల్వ/రేజ	స్థానం ఎంత తాజా
ప్రతి update ని DB lo రాయడం	2,50,000	0.86 TB	4 సెకన్లు
memory lo ఉంచి, 60 సెకన్లకి ఒకసారి DB	16,667	0.06 TB	4 సెకన్లు
memory మాత్రమే · DB lo trip మార్పులే	83	<0.01 TB	4 సెకన్లు

(మూడింటిలోనూ స్థానం ఒకేలా తాజాగా ఉంది - ఎందుకంటే *చదివేది* memory నుంచే)

2,50,000 → 83 · 3,000 రెట్లు · మరియు తాజాదనం ఒక్క సెకను కూడా తగ్గలేదు

చివరి వరుస చాలా ముఖ్యం: DB lo రాసేది **trip స్థితి మారినప్పుడు మాత్రమే** (బయలుదేరాడు, ఎక్కించాడు, దించాడు). అది ఒక్కే ride కి 3-4 writes.

మరియు అది **సరైన డేటా** - trip యొక్క చరిత్ర నిజంగా శాశ్వతంగా కావాలి (చెల్లింపులు, వివాదాలు, నియంత్రణ).

మీరు రాయాల్సినది "ఎక్కడ ఉన్నాడు" కాదు, "ఏమి జరిగింది".

Part 3 – రెండో విరుపు: వెంటనే కేటాయించడం

5. Step – దగ్గరివాడిని వెంటనే

Rider అడిగాడు. దగ్గరి ఖాళీ driver ని వెంటనే పంపడం – ఇది స్పష్టంగా సరైనది అనిపిస్తుంది:

```
function onRequest(rider) {  
  const driver = nearestFreeDriver(rider.x, rider.y);  
  offer(driver, rider);  
}
```

వేగం అత్యుత్తమం (rider ఒక్క క్షణం కూడా ఆగడు), మరియు ప్రతి rider కీ **ఆ క్షణంలో** అత్యంత దగ్గరి driver దొరుకుతాడు.

ఇందులో ఏమి తప్పు?

6. రెండో విరుపు – అత్యాశ ఎప్పుడు తప్పు అవుతుంది

"అత్యాశ" (greedy) యొక్క సమస్య ఇది: **ఇప్పటి rider కీ అత్యుత్తమమైనది, తర్వాతి rider కీ ఘోరం కావచ్చు.**

Driver A ఇద్దరు riders మధ్యలో ఉన్నాడు. మొదటి rider అడిగినప్పుడు A ని ఇచ్చేస్తాం. రెండో rider కీ ఇప్పుడు చాలా దూరంలో ఉన్న driver మిగిలాడు.

5-10 సెకన్లు ఆగి **ఇద్దరినీ కలిపి** కేటాయిస్తే మొత్తం దూరం తక్కువ అవుతుంది.

అది ఎంత తక్కువ? కొలిచి చూద్దాం – **మరియు నా మొదటి అంచనా తప్పు అయింది.**

ఒక నగరం 10x10 • 30 సెకన్లలో వచ్చే riders • supply ఎంత గట్టిగా ఉంది?

riders/drivers	సరఫరా	విధానం	సగటు pickup	కేటాయించినవి	అదనపు
ఎదురుచూపు					
-----+-----+-----+-----+-----+-----					
0.0 s	50 / 200	చాలా ఎక్కువ	అత్యాశ	0.400	50
		10 సె కిటికీ	0.396	50	
		మెరుగుదల	0.9%		
5.5 s					
0.0 s	100 / 120	కొంచెం ఎక్కువ	అత్యాశ	0.863	100

5.3 s		10 సె కిటికీ	0.836	100	
		మెరుగుదల	3.1%		
200 / 100	తక్కువ	అత్యాశ	1.384	100	
0.0 s		10 సె కిటికీ	0.830	100	
5.1 s		మెరుగుదల	40.1%		
400 / 100	చాలా తక్కువ	అత్యాశ	1.367	100	
0.0 s		10 సె కిటికీ	0.673	100	
5.1 s		మెరుగుదల	50.7%		

మొదటి వరుస - కిటికీ 0.9% మాత్రమే మెరుగుపరిచింది

Drivers ఎక్కువగా ఉన్నప్పుడు (50 riders కి 200 drivers), అత్యాశ ఇప్పటికే దాదాపు అత్యుత్తమం. కిటికీ పెట్టి rider ని 5.5 సెకన్లు ఆపి, 0.9% సాధించాం.

అది ఒక చెడ్డ బేరం. ఆ 5.5 సెకన్లు user కి కనిపిస్తాయి; ఆ 0.9% కనిపించదు.

నేను ఈ కొలతని "కిటికీ ఎప్పుడూ మంచిది" అని చూపిస్తుందని ఆశించాను. అది చూపించలేదు.

మూడో మరియు నాలుగో వరుసలు - అదే కిటికీ, 40% మరియు 50.7%

Drivers కొరతగా ఉన్నప్పుడు (200 riders కి 100 drivers), **ఎవరికి ఎవరిని ఇవ్వాలి** అనే క్రమం చాలా ముఖ్యం అవుతుంది - ఎందుకంటే ప్రతి తప్పు కేటాయింపు ఒక driver ని వృథా చేస్తుంది.

అదే 5 సెకన్ల ఎదురుచూపు ఇప్పుడు pickup దూరాన్ని **సగానికి** తగ్గిస్తుంది.

మరియు ఇది సరిగ్గా **మీకు అత్యంత అవసరమైన క్షణం** - peak hour, వర్షం, ఒక match ముగిసిన వేళ.

అదే 10-సెకన్ల కిటికీ - సరఫరా ని బట్టి పూర్తిగా వేరే ఫలితం

drivers చాలా ఎక్కువ (50 riders / 200 drivers)

0.9% మెరుగుదల - 5.5 సెకన్ల ఖర్చు - చెడ్డ బేరం

drivers తక్కువ (200 riders / 100 drivers)

40.1% మెరుగుదల - అదే 5 సెకన్లు

drivers చాలా తక్కువ (400 / 100)

50.7% మెరుగుదల

కిటికీ ఒక స్థిర నిర్ణయం కాకూడదు - అది సరఫరా ని బట్టి మారాలి.

7. Step – ఒక కిటికీ, ఒక గుంపు

```
// §7 – గుంపుగా కేటాయించడం. ఇక్కడే "అత్యాశ vs కిటికీ" తేడా.
tick() {
  const now = this.#clock();
  this.#expireOffers(now);
  const due = this.#pending.filter(id => {
    const r = this.#rides.get(id);
    return r && r.state === 'pending' && now - r.at >= this.batchMs;
  });
  if (!due.length) return 0;
  // ఈ గుంపు అంతటికీ (rider, driver) జతలు · దగ్గరివాటి నుంచి
  const free = [...this.#drivers.values()].filter(d => !d.busyWith);
  const pairs = [];
  for (const id of due) { const r = this.#rides.get(id);
    for (const d of free)
      pairs.push({ r, d, dist: Math.hypot(r.x-d.x, r.y-d.y) });
  }
  pairs.sort((a,b) => a.dist - b.dist);
  ...
}
```

`batchMs = 0` అయితే ఇది సరిగ్గా అత్యాశ. కాబట్టి **రెండు ఒకే code**, ఒక parameter తేడా.

అందుకే ఇది ఒక **PARAMETER** గా ఉండాలి, రెండు **CODEPATHS** గా కాదు

§6 పట్టిక చెప్పేది: సరైన `batchMs` అనేది సరఫరా-డిమాండ్ నిష్పత్తి యొక్క **function**.

కాబట్టి నిజమైన system దాన్ని **ప్రతి ప్రాంతానికీ, ప్రతి క్షణానికీ** మార్చాలి: drivers ఎక్కువగా ఉంటే 0, కొరతగా ఉంటే 5-10 సెకన్లు.

ఇది HLD Deep 02 §8 యొక్క అదే ఆలోచన – **స్థిర quota కంటే డిమాండ్ ని బట్టి మారేది మెరుగు**.

Part 4 – మూడో విరుపు: driver వద్దంటాడు

8. Step – offer పంపి ఎదురుచూడటం

Matching తెలివిగా ఉంది. కానీ \$1 lo చెప్పిన విషయం ఇక్కడ వస్తుంది: మనం కేటాయించడం లేదు, అడుగుతున్నాం.

Driver కి offer వెళ్తుంది. అతను 15 సెకన్లలో స్పందించాలి. అతను:

- అంగీకరించు – పని అయిపోయింది
- తిరస్కరించు – తర్వాతివాడికి వెళ్ళాలి
- ఏమీ చెప్పకపోవచ్చు – 15 సెకన్లు వృథా

ఆ మూడోది అత్యంత ఖరీదైనది.

9. మూడో విరుపు – 29 సెకన్లు, లేదా 2.5 మోసాలు

Driver ఒక request కి 15 సెకన్ల లోపు స్పందించాలి • 8 drivers పరిధిలో ఉన్నారు

అంగీకార రేటు | విధానం | సగటు ఎదురుచూపు | match కాలేదు | ఒక ride కి offers |
వృథా అయిన అంగీకారాలు

0.00	70% ఒక్కొక్కరిగా	5.0 s	0.0%	1.4	
1.16	70% ఒకేసారి 3 మందికి	3.4 s	0.0%	3.1	
2.52	70% ఒకేసారి 5 మందికి	2.8 s	0.0%	5.0	
0.00	40% ఒక్కొక్కరిగా	16.0 s	1.7%	2.4	
0.53	40% ఒకేసారి 3 మందికి	6.9 s	0.9%	3.8	
1.17	40% ఒకేసారి 5 మందికి	4.1 s	0.6%	5.4	
0.00	20% ఒక్కొక్కరిగా	29.2 s	16.4%	4.2	
0.20	20% ఒకేసారి 3 మందికి	12.2 s	13.5%	5.3	

20% అంగీకారం • ఒక్కొక్కరిగా అడిగితే – 29.2 సెకన్లు, మరియు 16.4% RIDERS కి ఎవరూ దొరకరు

కారణం సూటిగా ఉంది: 8 drivers lo ప్రతి ఒక్కరూ 15 సెకన్ల timeout తీసుకుంటారు. నలుగురు మౌనంగా ఉంటే అది **60 సెకన్లు**.

మరియు rider ఆ సమయంలో **ఏమీ జరగడం లేదు** అని చూస్తున్నాడు. అతను app మూసేస్తాడు.

పరిష్కారం స్పష్టం: **ఒకేసారి చాలామందికి పంపడం**. 20% అంగీకారం దగ్గర అది **29.2 → 7.0 సెకన్లు** (4 రెట్లు).

కానీ చివరి నిలుపు వరుస చూడండి.

70% అంగీకారం • ఒకేసారి 5 మందికి – ఒక RIDE కి 2.52 వృథా అంగీకారాలు

అంటే: ఐదుగురు drivers "అవును" అని నొక్కారు. ఒక్కరికే ride దొరికింది. మిగతా **2.52 మందికి** "క్షమించండి, ఇది ఇంకొకరు తీసుకున్నారు" అని చూపిస్తాం.

ఆ driver తన పని ఆపి, ఫోన్ తీసి, నొక్కాడు – మరియు ఏమీ రాలేదు. **అది రోజుకి పదిసార్లు జరిగితే అతను అంగీకరించడం మానేస్తాడు.**

మరియు అప్పుడు మే అంగీకార రేటు పడిపోతుంది – **మీరే మే సమస్యని తయారు చేసుకున్నాం.**

సరైన జవాబు – BROADCAST వెడల్పు అంగీకార రేటుకి విలోమంగా ఉండాలి

అంగీకారం **ఎక్కువ** (70%) → **ఒక్కొక్కరిగా** అడగండి. ఎదురుచూపు ఎలాగూ 5 సెకన్లే, మరియు ఎవరినీ మోసం చేయరు.

అంగీకారం **తక్కువ** (20%) → **ఒకేసారి ఐదుగురికి**. వృథా అంగీకారాలు 0.44 మాత్రమే (ఎందుకంటే ఎలాగూ చాలామంది వద్దంటారు), కానీ ఎదురుచూపు 29 నుంచి 7 సెకన్లకి.

ఇది §7 యొక్క అదే ఆకారం: **ఒక స్థిర నియమం కాదు – పరిస్థితిని బట్టి మారే నియమం.**

Part 5 – పూర్తి system

10. మొత్తం code • 11.7 లక్షల offers • mutation testing

నాలుగు నియమాలు

#	నియమం	విరిగితే అర్థం
1	ఒక driver కి ఒకసారి ఒకే ride	ఇద్దరు riders ఒకే car lo
2	busyWith మరియు ride స్థితి సరిపోవాలి	driver ఖాళీ అయ్యాడని system కి తెలియదు
3	స్థితి మార్పులు చెల్లుబాటులో ఉండాలి	pending నుంచి నేరుగా ontrip
4	అంగీకరించినవాడికి నిజంగా offer వెళ్ళి ఉండాలి	ఎవరైనా ఏ ride నైనా తీసుకోగలరు

నియమం 4 §9 నుంచి వస్తుంది: ఒకసారి ఐదుగురికి పంపినప్పుడు, **మొదట వచ్చినవాడికే** ఇవ్వాలి, మరియు మిగతావాళ్ళు అంగీకారం **పాతది** కావాలి.

```
// §10 – ఒకసారి చాలామందికి పంపాం; మొదట వచ్చినవాడికే. మిగతావాళ్ళది *పాతది*.
if (r.state !== 'offered' || !r.offers.has(driverId)) {
  this.stats.staleAccept++; return { ok: false, reason: 'T00_LATE' };
}
```

1,000 యాదృచ్ఛిక ప్రయోగాలు • 1,43,990 requests • 11,69,866 offers
నియమ ఉల్లంఘనలు: 0

ఏ దారులు నడిచాయి:

located	1,70,287
requested	1,43,990
offered	11,69,866
accepted	4,039
rejected	3,757
expiredOffer	11,26,690
assigned	4,039
completed	537
cancelled	32,405
staleAccept	1,15,130
badTransition	83,250

staleAccept 1,15,130 – అంటే §9 lo చూసిన "ఆలస్యంగా వచ్చిన అంగీకారం" 1.15 లక్షల సార్లు జరిగింది, మరియు ప్రతిసారీ సరిగ్గా తిరస్కరించబడింది.

Test విఫలం కాగలదా? – నాలుగు మార్పులు

మార్పు లేని code → 0/1000 విఫలం

"ఈ driver కి offer ఉందా" పరీక్ష తీసేస్తే → 285/300 విఫలం

ఉదా: driver d9 కి offer రాకుండానే ride 4 ని తీసుకున్నాడు
driver busy అని చూడకపోతే → 13/300 విఫలం

ఉదా: driver d4 కి రెండు rides: 9, 10
ride ముగిశాక driver ని ఖాళీ చెయ్యకపోతే → 211/300 విఫలం

ఉదా: ride 1 cancelled కానీ driver ఇంకా busy
స్థితి మార్పు నియమాలు పట్టించుకోకపోతే → 300/300 విఫలం

ఉదా: ride 1 ontrip కానీ driver లేడు

మొదటి మార్పు ముందు 0/300 – ఎందుకంటే నా పరీక్షలో ఒక నియమం లేదు

"ఈ driver కి offer వెళ్ళిందా" అనే పరీక్షని తీసేసినప్పుడు, ఏ driver అయినా ఏ ride నైనా తీసుకోగలడు. కానీ నా మొదటి మూడు నియమాలు అది **పట్టుకోలేదు** – ఎందుకంటే ఆ driver ఖాళీగా ఉంటే, "ఒక driver ఒక ride" ఇంకా నిజమే.

కావాల్సినది ఒక **మూలం** నియమం: "ఈ ride ఇతనికి ఎవరు ఇచ్చారు?"

దాన్ని test lo **విడిగా నమోదు** చేసి (system ని నమ్మకుండా) పరీక్షించాక – **285/300**.

"ఇది జరగకూడదు" అనే నియమాలు సరిపోవు. "ఇది ఎక్కడి నుంచి వచ్చింది" అనే నియమం కూడా కావాలి.

రెండో మార్పు **13/300** మాత్రమే – ఎందుకంటే driver busy ఉండగా అతనికి offer వెళ్ళే పరిస్థితి అరుదు (tick ఖాళీ drivers ని మాత్రమే చూస్తుంది). అది **ఒక రక్షణ పొర**, మరియు అరుదుగా అవసరమయ్యే రక్షణలే అత్యంత ముఖ్యమైనవి.

Part 6 – Interview lo

11. ఇదంతా 45 నిమిషాల్లో ఎలా చెప్పాలి

నిమిషాలు	ఏమి చెప్పాలి
0-5	"ఇది కేటాయింపు కాదు, ప్రతిపాదన" – driver వద్దనొచ్చు. ఇది మిగతా అందరి కంటే వేరుగా ఉంటుంది
5-9	§2 ప్రశ్నలు. ముఖ్యంగా "drivers ఎంత శాతం అంగీకరిస్తారు?"
9-15	మొదటి విరుపు. 2,50,000 writes/సె → 83. "స్థానం ఒక రికార్డు కాదు"
15-21	Geo index – geohash/H3, ఒక వాక్యం. లోతుకి వెళ్ళొద్దు (LLD Deep 11)
21-31	రెండో విరుపు. అత్యాశ vs కిటికీ – 0.9% vs 50.7%, సరఫరా ని బట్టి
31-40	మూడో విరుపు. 20% అంగీకారం → 29 సెకన్లు. Broadcast → 7 సెకన్లు, కానీ 2.5 మోసాలు
40-45	నాలుగు నియమాలు, ముఖ్యంగా "ఈ ride ఇతనికి ఎవరు ఇచ్చారు?"

ఏమి తప్పక చెప్పాలి

1. Driver ఒక వనరు కాదు, ఒక వ్యక్తి. అతన్ని కేటాయించలేం, అడగగలం.
2. స్థానం ఒక క్షణిక స్థితి, ఒక రికార్డు కాదు. 2,50,000 → 83 writes.
3. అత్యాశ ఎప్పుడు తప్పో సరఫరా నిర్ణయిస్తుంది – 0.9% లేదా 50.7%.
4. Broadcast వెడల్పు అంగీకార రేటుకి విలోమంగా. ఎక్కువమందికి పంపడం drivers ని మోసం చేస్తుంది.
5. ఈ రెండు నిర్ణయాలూ (కిటికీ, వెడల్పు) స్థిరంగా ఉండకూడదు – పరిస్థితిని బట్టి మారాలి.

ఏమి వదిలేయాలి

- Geohash/H3 యొక్క గణితం – ఒక వాక్యం (LLD Deep 11 lo ఉంది)
- Map matching, routing, ETA యొక్క algorithms – §13 lo ఒక వాక్యం
- Payment – వేరే problem (HLD Deep 09)
- Driver rating, background checks

12. నీటితో చెప్పాల్సిన English script

"One thing distinguishes this from every resource-allocation problem I've designed: **a driver can say no**. A connection pool hands you a connection; a thread pool hands you a thread. Here I can only make an **offer**, and I measured acceptance rates as low as 20%. So my 'optimal assignment' is a suggestion – the real allocation happens on drivers' phones."

"First, the location firehose. A million drivers reporting every four seconds is **250,000 writes per second**, 21.6 billion rows a day, and **nobody ever reads any of them** except the latest one per driver. More importantly, losing that data is fine – every driver re-reports within four seconds. **Location isn't a record, it's ephemeral state**, so it belongs in memory. Persist trip state transitions instead – picked up, dropped off – which is 83 writes per second, a **3,000x reduction**, with identical freshness because reads come from memory either way. And the data you *do* persist is the data you actually need, for payments and disputes."

"For the geo index I'd use geohash or H3 and move on – I've measured that elsewhere and it's not where this problem is hard."

"Second, matching. The obvious rule is 'assign the nearest free driver immediately'. The known objection is that it's greedy – assigning a driver who sits between two riders hurts the second one. So I measured batching over a ten-second window, expecting a clear win. **I didn't get one**. With drivers plentiful – 50 riders, 200 drivers – batching improved pickup distance by **0.9%** for 5.5 seconds of added wait. That's a bad trade; the wait is visible and the gain isn't."

"But with drivers scarce – 200 riders, 100 drivers – the same window improved it **40%**, and at 400 riders **50.7%**. Because when supply is tight, every wrong assignment wastes a driver. So the answer isn't 'always batch' or 'never batch': **the window should be a function of the supply-demand ratio**, near zero when drivers are plentiful and five to ten seconds during surge. Which is exactly when it matters."

"Third, drivers declining. At 20% acceptance, asking one at a time means each silent driver burns a 15-second timeout: average wait **29.2 seconds**, and **16.4% of riders never get matched** – they close the app. Broadcasting to five drivers cuts that to **7 seconds**."

"But there's a cost people skip. At 70% acceptance, broadcasting to five produces **2.52 wasted acceptances per ride** – two and a half drivers tapped 'accept', stopped what they were doing, and got 'sorry, taken'. Do that ten times a day and they stop accepting, which **lowers the acceptance rate you were compensating for**. So broadcast width should be **inverse to acceptance rate**: one at a time when drivers are eager, five when they aren't. Same shape as the batching window – a parameter that reads the situation, not a fixed rule."

"Four invariants, a thousand runs, 1.17 million offers, zero violations. The interesting one is invariant four: **the driver who accepted must actually have been offered it**. My first three invariants missed a mutation that let any driver accept any ride, because if that driver was free, 'one driver one ride' still held. I had to record independently, in the test, who was offered what. **'This must not happen' invariants aren't enough – you also need 'where did this come from'.**"

13. Follow-ups – surge, pooling, ETA

"Surge pricing ఎక్కడ కూర్చుంటుంది?"

§6 యొక్క అదే సంఖ్య – **సరఫరా-డిమాండ్ నిష్పత్తి** – surge ని కూడా నడిపిస్తుంది. అంటే మీ dispatcher ఇప్పటికే ఆ సంఖ్యని లెక్కిస్తోంది. ముఖ్యమైన design విషయం: surge **ప్రాంతాల వారీగా** ఉండాలి, మరియు ఆ ప్రాంతాలు **చిన్నవిగా** ఉండాలి – లేకపోతే ఒక చోట ఉన్న కొరత మొత్తం నగరం ధరని పెంచుతుంది. మరియు ఒక అసక్తికరమైన అభిప్రాయం: surge drivers ని ఆకర్షించి **సరఫరా పెంచుతుంది**, కాబట్టి అది §6 యొక్క నిష్పత్తిని మారుస్తుంది – మీ కిటికీ కూడా దానితో మారాలి.

"Pooling (ఇద్దరు riders ఒకే car lo)?"

నా నియమం 1 – "ఒక driver కి ఒకే ride" – **మారుతుంది**. అది "ఒక driver కి ఒకే మార్గం" అవుతుంది, మరియు ఒక మార్గంలో చాలా riders ఉండొచ్చు. కొత్త కష్టం: **ఒక కొత్త rider ని చేర్చడం ఇప్పటికే car lo ఉన్నవాళ్ళ ప్రయాణాన్ని పొడిగిస్తుంది**, కాబట్టి కేటాయింపు నిర్ణయం ఇప్పుడు "దూరం" కాదు, "అదనపు దూరం ÷ ఆదా అయిన car". §7 యొక్క కిటికీ ఇక్కడ **ఇంకా ముఖ్యం** – pooling కి మీకు ఒకేసారి చాలా riders కనిపించాలి.

"ETA ఎలా లెక్కించాలి?"

నేరుగా దూరం చాలదు – traffic, signals, one-way roads. ఆచరణలో ఇది ఒక ప్రత్యేక service (road graph + చారిత్రక traffic + ప్రస్తుత వేగాలు). ఒక design విషయం: **dispatch కి ETA కావాలి, కానీ ETA ఖరీదు**. §7 lo నేను అన్ని (rider, driver) జతలకీ దూరం లెక్కించాను – నిజమైన ETA తో అది వేలాది API calls అవుతుంది. కాబట్టి ఆచరణ: **ముందు నేరుగా దూరంతో top-k వడపోసి, ఆ k కి మాత్రమే నిజమైన ETA**.

"Trip మధ్యలో ఫోన్ network పోతే?"

State machine యొక్క **సత్యం server మీద** ఉండాలి, ఫోన్ మీద కాదు. ఫోన్ తన స్థితిని పంపుతుంది, కానీ server దాన్ని **చెల్లుబాటు చేస్తుంది** (§10 యొక్క **LEGAL** పట్టిక). నెట్‌వర్క్ తిరిగి వచ్చినప్పుడు ఫోన్ server నుంచి నిజమైన స్థితిని తెచ్చుకుంటుంది. మరియు ఒక ఆచరణాత్మక విషయం: **driver "దించాను" అని నొక్కినప్పుడు network లేకపోతే**, ఆ చర్యని ఫోన్ lo queue చేసి తర్వాత పంపాలి – కానీ దాని **సమయాన్ని** ఫోన్ గడియారం నుంచి కాదు, server assign చేసిన sequence నుంచి తీసుకోవాలి (HLD Deep 04 §5 అదే పాఠం).

14. ఏమి నేర్చుకున్నాం

1. Driver ఒక వనరు కాదు, ఒక వ్యక్తి. Connection pool lo **borrow()** వద్దనలేదు; ఇక్కడ 80% సార్లు వద్దనొచ్చు. అది మీ మొత్తం design ని మారుస్తుంది.

2. స్థానం ఒక రికార్డు కాదు, ఒక క్షణిక స్థితి. పోయినా 4 సెకన్లలో తిరిగి వస్తుంది – కాబట్టి database lo ఉండాలైన అవసరం లేదు. **2,50,000 → 83 writes**, తాజాదనం అదే.

3. "ఏమి జరిగింది" రాయండి, "ఎక్కడ ఉన్నాడు" కాదు. మొదటిది శాశ్వతమైన నిజం; రెండోది 4 సెకన్ల ఉజ్జాయింపు.

4. అత్యాశ ఎప్పుడు తప్పో పరిస్థితి నిర్ణయిస్తుంది. Drivers ఎక్కువగా ఉంటే కిటికీ **0.9%** – వృథా. కొరతగా ఉంటే **50.7%** – అత్యవసరం. నా అంచనా తప్పు అయింది, మరియు కొలత దాన్ని సరిచేసింది.

5. ఒక సమస్యకి పరిష్కారం ఇంకొక సమస్యని తయారు చేయొచ్చు. Broadcast ఎదురుచూపుని 29 నుంచి 7 సెకన్లకి తగ్గించింది – మరియు **2.52 drivers ని మోసం చేసింది**, ఇది వాళ్ళ అంగీకార రేటుని తగ్గిస్తుంది, ఇది మళ్ళీ ఎదురుచూపుని పెంచుతుంది.

6. రెండు ముఖ్య parameters (కిటికీ, broadcast వెడల్పు) స్థిరంగా ఉండకూడదు. రెండూ ఒకే విషయం చదవాలి: **ఇప్పుడు పరిస్థితి ఎలా ఉంది.**

7. "ఇది జరగకూడదు" నియమాలు సరిపోవు. ఏ driver అయినా ఏ ride నైనా తీసుకోగలిగినా నా మొదటి మూడు నియమాలు **మౌనంగా ఉన్నాయి**. కావాల్సింది ఒక **మూలం** నియమం: "ఇది ఇతనికి ఎవరు ఇచ్చారు?" – అది జోడించాక **285/300**.

ఈ **DOC** నుంచి ఒక్క వాక్యం గుర్తుపెట్టుకోవాలంటే

ఈ **system** యొక్క రెండు కీలక నిర్ణయాలూ – ఎంతసేపు ఆగాలి, ఎంతమందిని అడగాలి – రెండూ **"ఇప్పుడు పరిస్థితి ఎలా ఉంది"** అనే ఒకే ప్రశ్నకి జవాబులు.

Drivers ఎక్కువగా ఉంటే: ఆగవద్దు, ఒక్కొక్కరినే అడగండి.

Drivers కొరతగా ఉంటే: ఆగండి, చాలామందిని అడగండి.

ఒకే code, రెండు parameters – మరియు ఆ రెండింటినీ **స్థిరంగా ఉంచడమే** ఈ problem lo అతి సాధారణమైన తప్పు.